UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II 2025/2026

(SECP3133) HIGH PERFORMANCE DATA PROCESSING

SECTION 02

PROJECT 1: OPTIMIZING HIGH-PERFORMANCE DATA PROCESSING FOR LARGE-SCALE WEB CRAWLERS

LECTURER: DR. SEAH CHOON SEN

STUDENT NAME	MATRIC NO
GUI KAH SIN	A23CS0080
SABRINA HENG WEI QI	A23CS0265
LING YU QIAN	A23CS0301
WOO CHENG SHUAN	A23CS0283

Table of Contents

1.0 Introduction

1.1. Background

Internet has become one of the largest sources of information in modern society. Every day, websites will generate and publish massive amounts of content including news articles, advertisements, social media updates and other forms of information. As the volume of online data rapidly grows, automated techniques are required to efficiently collect and process the information.

Web crawling is a technique used to automatically retrieve information from the websites. A function web crawler will systematically visit web pages, extract relevant data and store the collected information in a structured format for further business use or analysis. Therefore, web crawlers significantly reduce the time and effort to gather huge datasets compared to manual data ingestion.

However, large-scale web crawling may face several challenges. Modern websites often employ dynamic content loading mechanisms such as AJAX (Asynchronous JavaScript and XML), which means that data is not always directly available in the initial HTML page. Instead, additional information is loaded dynamically through background requests after user interactions. Consequently, traditional crawling approaches may be inadequate and require a deeper understanding of how websites retrieve and display data.

High-performance Computing (HPC) techniques can be applied to improve the efficiency of large-scale web crawling systems. By optimizing the ingestion process, larger datasets can be collected within shorter periods while maintaining system reliability and scalability. Thus, this project focuses on developing a web crawler capable of extracting structured data from a Malaysian technology news website, named “SoyaCincau”. The collected data serves as the foundation for subsequent data processing, optimization and performance evaluation activities.

1.2. Objectives

The objectives of this project are:

i. To develop an automated web crawler capable of extracting structure information from a Malaysian website.
ii. To identify and utilize the website's AJAX-based data retrieval mechanism.
iii. To collect article metadata including title, category, publication date, URL and article summary.
iv. To store extracted data in a structured JSON format for future use and analysis.
v. To establish a baseline crawling system that can later be optimized using High-Performance Computing techniques.

1.3. Target Website and Data to be Extracted

SoyaCincau was selected as the target website for this project. SoyaCincau is one of the Malaysia's leading technology-focused news portals and regularly publishes articles related to smartphones, telecommunications, electric vehicles, artificial intelligence, digital services and customer technology.

There are several factors influenced the selection of this website. First, the website publishes a large volume of articles regularly, making it a good source for data collection. Besides, all articles information can be accessed by the public without requiring user authentication, allowing data to be collected without violating access restrictions. Moreover, the layout and structure of the articles are consistent, making it easier to extract information from them.The website also provides useful details that can be used for further analysis. For instance, the crawler was designed to extract information such as the article title, category, publication date, URL and summary.

Initial investigation also revealed that the website utilizes AJAX-based dynamic content loading to retrieve additional articles through a "Load More" mechanism. This characteristic provides an opportunity to study modern web crawling techniques beyond traditional page-based scraping. The structured nature of the website and its dynamic content delivery mechanism make SoyaCincau a suitable and good platform for developing and evaluating a scalable web crawling system.

2.0 System Design & Architecture

2.1 System Overview

The system developed in this project consists of four major phases which is data collection, data cleaning, high-performance optimization and performance evaluation. The overall architecture was designed to support efficient extraction and processing of structured web data obtained from the SoyaCincau technology news website.

The workflow begins with the web crawler, which communicates directly with SoyaCincau's AJAX-based content loading mechanism to retrieve article information. The extracted records are progressively stored in JSON format to prevent data loss during the crawling process. The collected raw dataset is then forwarded to the data cleaning pipeline, where validation, transformation and standardization operations are performed.

After data cleaning, the processed dataset serves as the input for the High-Performance Computing (HPC) optimization phase. Multiple processing strategies including sequential processing, multithreading, multiprocessing and distributed processing using Dask are applied to evaluate different execution models. Finally, performance metrics are collected and analysed to determine the effectiveness of each optimization technique.

Figure 2.1 Overall System Architecture

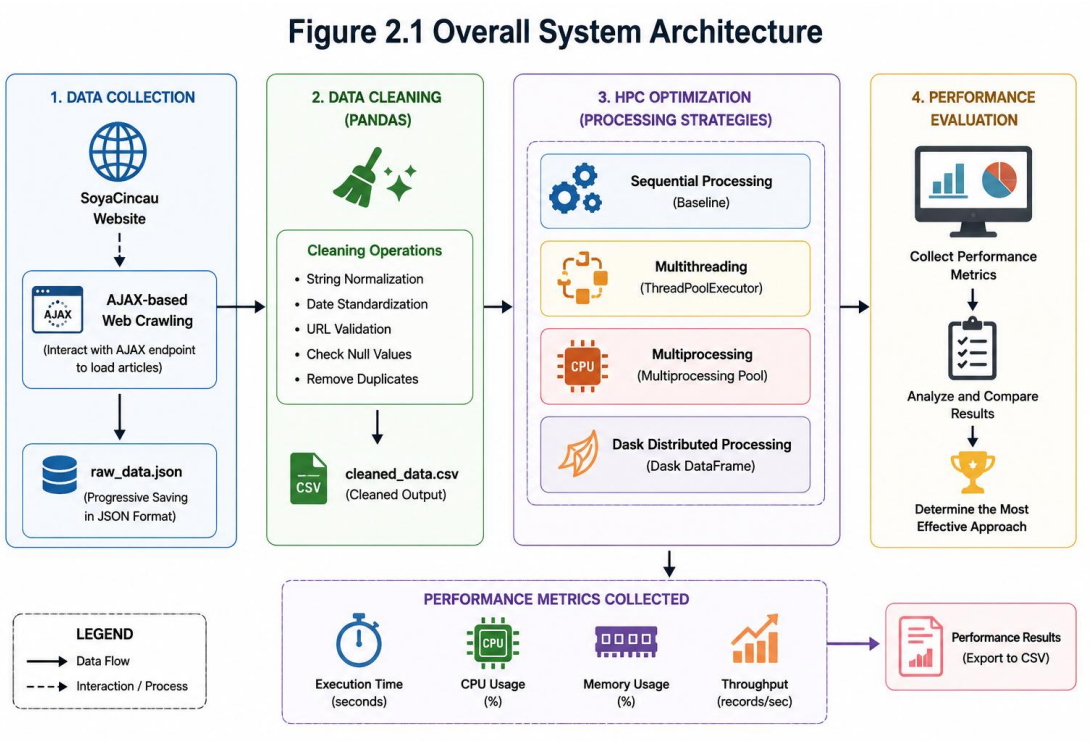

Figure 2.1 Overall System Architecture

2.2 System Components

The architecture shown in Figure 2.1 consists of the following major components:

- Web Crawler

  The web crawler is responsible for retrieving article information from the SoyaCincau website. The crawler interacts with the website's AJAX endpoint to collect article metadata. The extracted records are stored progressively in JSON format to ensure data persistence throughout the crawling process.
- Data Cleaning Module

  The data cleaning module processes the raw dataset generated by the crawler. The cleaned output is exported as a CSV file and serves as the input for subsequent optimization experiments.
- HPC Optimization Module

  The optimization module evaluates different processing approaches for handling the cleaned dataset. Three optimization techniques will be use in this module. These techniques introduce varying degrees of parallelism and resource utilization into the processing workflow.
- Performance Evaluation Module

  The performance evaluation module collects execution metrics from each processing approach. The generated metrics are subsequently analysed to determine processing efficiency, scalability and throughput performance.

2.3 Tools and Frameworks

In this project, there are several software libraries and frameworks that is used throughout the project. Table 2.1 shows that the table of software tools and framework used during project. These technologies can provide a complete environment for data collection, data processing, optimization and evaluation.

Table 2.1 Software Tools and Frameworks

Tool / Framework	**Purpose**
Python	Primary programming language
Requests	HTTP communication with website endpoints
BeautifulSoup	HTML parsing and data extraction

Pandas	Data cleaning and transformation
NumPy	Dataset partitioning
ThreadPoolExecutor	Multithreading implementation
Multiprocessing	Parallel CPU processing
Dask	Distributed data processing
JSON	Raw data storage format
CSV	Cleaned dataset storage format
Google Collab	Development and execution environment

2.4 Team Responsibilities

To ensure efficient project execution, responsibilities were distributed among team members according to different phases of the workflow. This division of responsibilities allowed each project component to be developed and evaluated independently while maintaining integration across the complete system workflow.

Table 2.2 Team Responsibilities

Member	**Responsibility**
Member 1 – GUI KAH SIN	Web crawler development and data collection
Member 2 – WOO CHENG SHUAN	Data cleaning pipeline and validation
Member 3 – SABRINA HENG WEI QI	HPC optimization and system architecture
Member 4 – LING YU QIAN	Performance evaluation and result analysis

3.0 Data Collection

3.1 Crawling Method

The data collection process was performed using a custom Python-based web crawler developed to extract article information from the SoyaCincau website. Initial investigation revealed that the website utilizes AJAX-based dynamic content loading instead of traditional page navigation. Additional articles are retrieved through background HTTP POST requests whenever users interact with the "Load More" functionality.

To efficiently collect data, the crawler directly communicates with the website's AJAX endpoint rather than repeatedly downloading entire webpages. The crawler sends POST requests containing pagination parameters and receives article information in JSON format. The returned HTML content is then parsed using BeautifulSoup to extract the required article attributes, such as article title, URL, category, publication date and article excerpt.

The crawler follows a pagination-based crawling approach by incrementally requesting multiple pages of article content. This method enables continuous data collection while maintaining an organized crawling process. Since the website does not require user authentication to access article information, all data was collected from publicly available sources.

Although asynchronous crawling and multithreading techniques were not implemented during the initial data collection stage, the crawler serves as a baseline system that can be further optimized using high-performance data processing techniques in subsequent phases of the project

3.2 Number of records collected

The crawler was executed across multiple AJAX pages available on the target website, SoyaCincau. During the crawling process, extracted article information was stored in JSON format for future processing and analysis.

A total of 30,828 article records were successfully collected from SoyaCincau. Each record contains structured metadata including article title, category, publication date, article URL and excerpt. The collected dataset provides sufficient data for subsequent stages of the project, including data cleaning, transformation, optimization and performance evaluation.

3.3 Ethical Considerations

Ethical web crawling practices were observed throughout the project. The crawler accesses only publicly available information and does not attempt to bypass authentication mechanisms or access restricted content. Furthermore, requests were issued in a controlled manner to minimize unnecessary load on the target website and prevent disruption of normal website operations.

The collected data is used solely for academic and research purposes. No personal information, user-generated content requiring authentication or restricted resources were collected during the crawling process. Adhering to responsible crawling practices helps ensure that data collection activities remain ethical, sustainable and respectful to the website operators.

4.0 Data Cleaning Pipeline

4.1 Overview

The data cleaning pipeline was designed to process the data obtained from the web crawler, making it more consistent and reliable for subsequent analysis. The pipeline processes raw data from a file (raw_data.json), cleans and validates the data and writes the cleaned data to a file (cleaned_data.csv). The cleaning of the data was performed in Python with the Pandas library and in the clean_data.ipynb file.

4.2 Raw Data Loading

The data was loaded from Google Drive into a Pandas Data Frame for further processing. The dataset was initially found to consist of 30,834 entries and five numerical attributes: *title, url, category, date* and *excerpt*. All attributes had been saved as string (object) data types, meaning the scraper didn't perform automatic conversion to the data type.

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
(30834, 5)
title       object
url         object
category    object
date        object
excerpt     object
dtype: object
```

Figure 2.1 Raw Data Shape and Column Types

As shown in Figure 2.1, the number of dimensions in the dataset and the type of data were verified, ensuring that data was successfully loaded and ready for cleaning.

4.3 Data Inspection

The dataset was reviewed for missing values and duplicate data prior to any cleaning. This step was crucial to assess the data overall quality and detect potential problems which could impact further analysis.

```
title       0
url         0
category    0
date        0
excerpt     0
dtype: int64
0
```

Figure 2.2 Null Value Count and Duplicate Check

During the inspection no missing values were found in any of the columns as displayed in Figure 2.2. There were no duplicate records found as well. The results show that during the crawling, the complete and no-overlapping records were obtained.

4.4 String Field Cleaning

The text-based attributes were subjected to several cleaning operations to ensure consistency and eliminate formatting inconsistencies that are often observed when scraping from the web and the intended use of the text-based attributes. The following transformations were carried out:

- Leading and trailing spaces have been trimmed off the title, excerpt, category and url fields.
- All multiple spaces or newlines in the title or excerpt were collapsed to a single space.
- Using the title case formatting, the category field was standardised so that the categories are named consistently.

These cleaning procedures ensured the data was more consistent and maintained the original meaning and content of each article.

```
String cleaning done
                                                title  \
0  No more RM1 interbank ATM fee from 1st July 20...
1  Perodua QV-E is now priced from RM63,499: Also...
2  Meta offers Instagram Plus, WhatsApp Plus and ...
3  iCaur Malaysia upgrades EV battery warranty to...
4  Vivo Y31s Malaysia: 6,500mAh battery, IP69+ du...

                                                 url      category  \
0  https://soyacincau.com/2026/06/15/banks-malays...  Digital Life
1  https://soyacincau.com/2026/06/15/perodua-qve-...            Ev
2  https://soyacincau.com/2026/06/15/meta-instagr...          Apps
3  https://soyacincau.com/2026/06/14/icaur-malays...          Cars
4  https://soyacincau.com/2026/06/14/vivo-y31s-5g...      Featured

            date                                            excerpt
0  June 15, 2026  While most Malaysians have gone cashless, here...
1  June 15, 2026  Perodua QV-E can now be obtained from as low a...
2  June 15, 2026  Meta has finally rolled out its paid subscript...
3  June 14, 2026  iCaur Malaysia has announced enhanced warranty...
4  June 14, 2026  Vivo Malaysia has introduced the new Vivo Y31s...
```

Figure 2.3 Sample Data After String Cleaning

In Figure 2.3 shows several sample records after the string cleaning process. The resulting dataset is cleaner, more structured and easier to process in subsequent analysis stages while maintaining the original information from the scraped articles.

4.5 Date Standardization

This publication date was initially recorded as a string of the form “June 15, 2026”. This format is human-readable but it’s not an ideal format to sort, filter or perform time-series analysis upon. To solve this problem, the date column had been changed to datetime data type by using pd.to_datetime(errors = “coerce”). The dates were then converted to ISO 8601 standard format (YYYY-MM-DD).

```
Unparseable dates: 0
Date cleaning done
0    2026-06-15
1    2026-06-15
2    2026-06-15
3    2026-06-14
4    2026-06-14
Name: date, dtype: object
```

Figure 2.4 Date Parsing Results and Standardized Format

Figure 2.4 shows the date parsing samples. There were no parsing errors and all 30,834 records were successfully converted since the data in the original text was formatted the same across the entire dataset.

4.6 URL Validation

The URL attribute was checked to make sure all records had an entire URL with the prefix of "http". This validation is useful in detecting links that are malformed or incomplete which can be captured during scraping.

Figure 2.5 URL Validation Result

No invalids URls were found in the validation process as shown in Figure 2.5. As such, all records were kept, verifying that each article record has a valid webpage address.

4.7 Final Validation and Index Reset

The index of the DataFrame was reset to generate continuous sequence of record identifiers after all the cleaning operations performed. A final cross-checking was then done to ensure that the cleaning process had not introduced any missing values.

```
Final record count: 30834
title       0
url         0
category    0
date        0
excerpt     0
dtype: int64
                                                title  \
0  No more RM1 interbank ATM fee from 1st July 20...
1  Perodua QV-E is now priced from RM63,499: Also...
2  Meta offers Instagram Plus, WhatsApp Plus and ...
3  iCaur Malaysia upgrades EV battery warranty to...
4  Vivo Y31s Malaysia: 6,500mAh battery, IP69+ du...

                                                 url      category  \
0  https://soyacincau.com/2026/06/15/banks-malays...  Digital Life
1  https://soyacincau.com/2026/06/15/perodua-qve-...            Ev
2  https://soyacincau.com/2026/06/15/meta-instagr...          Apps
3  https://soyacincau.com/2026/06/14/icaur-malays...          Cars
4  https://soyacincau.com/2026/06/14/vivo-y31s-5g...      Featured

         date                                            excerpt
0  2026-06-15  While most Malaysians have gone cashless, here...
1  2026-06-15  Perodua QV-E can now be obtained from as low a...
2  2026-06-15  Meta has finally rolled out its paid subscript...
3  2026-06-14  iCaur Malaysia has announced enhanced warranty...
4  2026-06-14  Vivo Malaysia has introduced the new Vivo Y31s...
```

Figure 2.6 Final Dataset Summary

As shown in Figure 2.6, the last dataset retained all 30,834 records and no missing data was found for any attribute. This means that the cleaning process did not result in any data being lost.

4.8 Data Dictionary

Table 2.1 shows the cleaned data dictionary with columns, data types, descriptions and examples.

Table 2.1 Cleaned Data Dictionary

Column	Data Type	Description	Example
Title	String	Article headline published on SoyaCincau	“No more RM1 interbank ATM fee from 1st July 20...”

URL	String	Direct URL linking to the article	"https://soyacincau.com/2026/06/15/banks-malays..."
Category	String	Category assigned to the article.	"Digital Life", "Ev", "Apps"
Date	String (ISO 8601)	Publication date in YYYY-MM-DD format	"2026-06-15"
Excerpt	String	Short summary of the article	"While most Malaysians have gone cashless, here..."

4.9 Output

Cleaned data was saved in the shared project folder of Google Drive as cleaned_data.csv. The output is 30,834 validated records, which are the main input to the High-Performance Computing (HPC) optimization pipeline developed and to the performance evaluation activities carried out by the following team members.

```
Saved cleaned_data.csv with 30834 records
```

Figure 2.7 Export Confirmation

Figure 2.7 shows that the dataset has been successfully exported and is ready to proceed with the next phase of the project workflow.

5.0 Optimization Techniques

5.1 Overview

After the completion of the data cleaning phase, the cleaned dataset containing 30,834 validated article records was used as the input for High-Performance Computing (HPC) optimization. The objective of this phase was to investigate how parallel and distributed processing techniques could improve the efficiency and scalability of the existing processing workflow.

A baseline sequential implementation was first established as a reference point. Subsequently, three optimization techniques were introduced, which are multithreading, multiprocessing and distributed processing using Dask. Each technique was implemented using the same dataset and identical cleaning operations to ensure consistency throughout the evaluation process.

5.2 Sequential Processing Baseline

The baseline implementation utilized a conventional sequential processing model. Under this approach, all cleaning operations were executed one after another within a single processing flow. No parallel execution or task distribution was performed.

The sequential implementation served as the benchmark against which all optimization techniques were compared. Although simple and easy to implement, this approach utilizes only a single execution path and may become inefficient when processing larger datasets.

5.3 Multithreading Implementation

The first optimization technique implemented in this project was multithreading. Python;s ThreadPoolExecutor was utilized to create multiple worker threads capable of processing different portions of the dataset concurrently.

The dataset was partitioned into four chunks using NumPy’s array splitting functionality. Each chunk was assigned to a separate worker thread, allowing cleaning operations to be performed simultaneously. Figure 5.1 presents the implementation of the multithreading approach.

```python
cpu_before = psutil.cpu_percent(interval=None)
memory_before = psutil.virtual_memory().percent

start = time.time()

with ThreadPoolExecutor(max_workers=4) as executor:

    results = list(
        executor.map(
            clean_chunk,
            chunks
        )
    )

thread_df = pd.concat(results)

thread_time = time.time() - start

cpu_after = psutil.cpu_percent(interval=None)
memory_after = psutil.virtual_memory().percent

thread_cpu = cpu_after
thread_memory = memory_after
```

Figure 5.1: Multithreading Implementation

By enabling concurrent execution, multithreading improves resource utilization and reduces idle waiting time. This approach is particularly effective for workloads involving significant input or output operations and lightweight processing tasks.

5.4 Multiprocessing Implementation

The second optimization technique implemented was multiprocessing. Unlike multithreading, multiprocessing creates multiple independent processes that execute simultaneously across separate CPU cores.

The cleaned dataset was divided into four partitions and distributed across four worker processes using Python's multiprocessing library. Each process independently executed the required cleaning operations before the results were merged into a single DataFrame.

```
cpu_before = psutil.cpu_percent(interval=None)
memory_before = psutil.virtual_memory().percent

start = time.time()

with Pool(4) as pool:

    results = pool.map(
        clean_chunk,
        chunks
    )

process_df = pd.concat(results)

process_time = time.time() - start

cpu_after = psutil.cpu_percent(interval=None)
memory_after = psutil.virtual_memory().percent

process_cpu = cpu_after
process_memory = memory_after
```

Figure 5.2 Multiprocessing Implementation

Multiprocessing enables true parallel execution because each worker process operates independently. This approach is particularly suitable for CPU-intensive workloads where computational tasks can be distributed across multiple processor cores.

5.5 Dask Distributed Processing

The third optimization technique implemented was distributed processing using Dask. Dask extends the functionality of Pandas by enabling datasets to be partitioned and processed concurrently.

The cleaned dataset was converted into a Dask DataFrame containing multiple partitions. Cleaning operations were applied independently to each partition before the final output was generated using the compute() function.

```
cpu_before = psutil.cpu_percent(interval=None)
memory_before = psutil.virtual_memory().percent

start = time.time()

ddf["title"] = (
    ddf["title"]
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
)

ddf["category"] = (
    ddf["category"]
    .str.strip()
    .str.title()
)

ddf["excerpt"] = (
    ddf["excerpt"]
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
)

dask_df = ddf.compute()

dask_time = time.time() - start

cpu_after = psutil.cpu_percent(interval=None)
memory_after = psutil.virtual_memory().percent

dask_cpu = cpu_after
dask_memory = memory_after
```

Figure 5.3 Dask Distributed Processing

Dask introduces a scalable processing model that supports larger datasets and distributed computing environments. Although the dataset used in this project was relatively moderate in size, Dask demonstrates how distributed computing concepts can be integrated into a web crawler data processing pipeline.

5.6 Optimized Processing Architecture

The optimization pipeline begins with the cleaned dataset generated from the data cleaning phase. The dataset is subsequently processed using sequential processing, multithreading, multiprocessing and Dask-based distributed processing. Performance metrics including execution time, CPU utilization, memory usage and throughput are collected during execution and exported for further analysis.

The implementation of multiple optimization techniques provides a practical demonstration of High-Performance Computing concepts and establishes a scalable foundation for future large-scale data processing applications.

6.0 Performance Evaluation

6.1 Benchmark Methodology

All four processing methods were benchmarked on an identical workload: applying the complete data cleaning transformation pipeline to the full 30,834-record dataset loaded from raw_data.json. Execution time was recorded using Python's time.time() function, measuring wall-clock time from the start of data processing to the completion of all transformations. Throughput was calculated as the total record count divided by execution time (records per second). All benchmarks were executed in Google Colab on a single-node CPU environment.

Performance metrics were captured and stored in performance_results.csv for subsequent visualization and statistical analysis. The evaluation notebook (evaluation_charts.ipynb) generated four charts: a Seaborn bar chart for execution time, a Seaborn horizontal bar chart for throughput, a Seaborn bar chart for speedup factors, and a composite Plotly interactive dashboard.

6.2 Results Summary Table

Table 4.1 presents the complete benchmark results for all four processing methods.

Table 4.1 Performance Benchmark Results (Dataset: 30,834 records)

Method	**Execution Time (s)**	**Throughput (rec/s)**	**Speedup vs Sequential**
Sequential	0.5706	54,042	1.000×
Multithreading	0.6419	48,037	0.889×
Multiprocessing	0.8095	38,091	0.705×
Dask	0.5235	58,905	1.090×

6.3 Chart Analysis

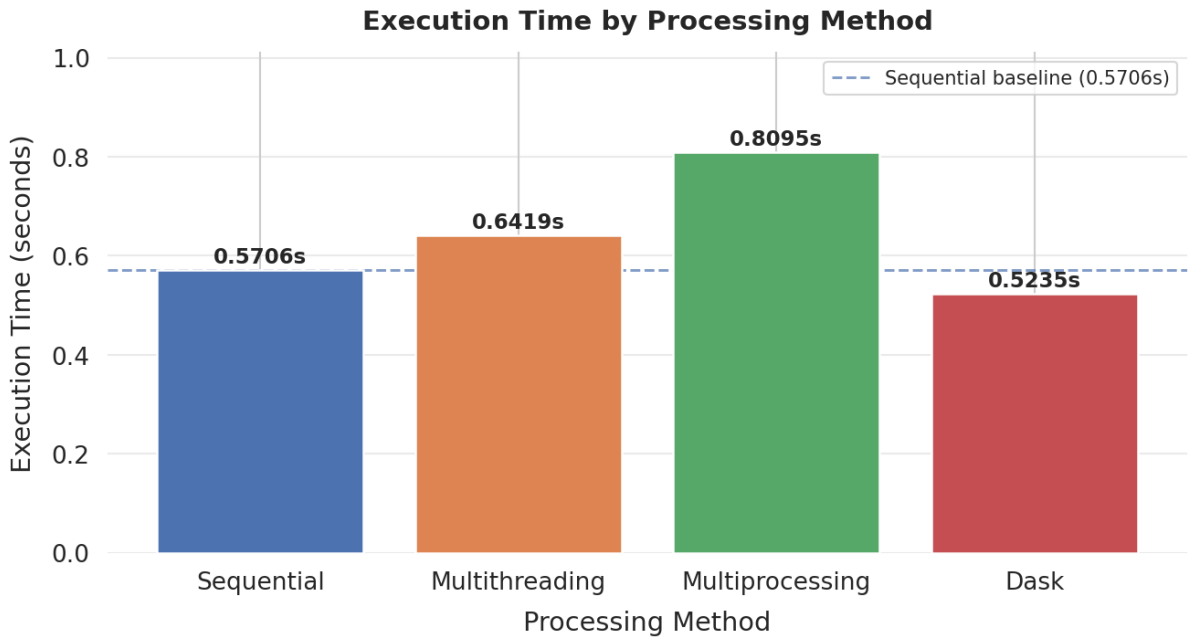

Chart 1 — Execution Time Comparison: The bar chart reveals that Dask achieves the shortest execution time at 0.5235 seconds, followed closely by Sequential (0.5706s). Multithreading (0.6419s) and Multiprocessing (0.8095s) both exceed the sequential baseline, demonstrating that parallelism overhead outweighs gains for this workload and dataset size.

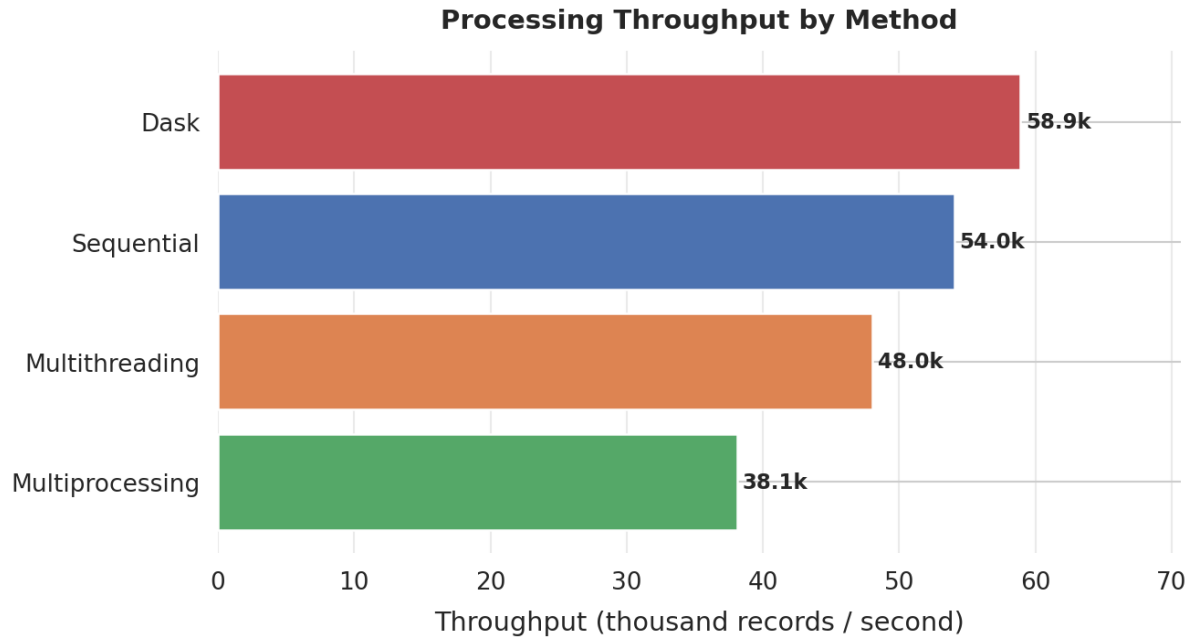

Chart 2 — Throughput Comparison: The horizontal bar chart, sorted from lowest to highest throughput, shows Dask leading at 58,905 records/second, followed by Sequential (54,042), Multithreading (48,037), and Multiprocessing (38,091). The spread between the highest and

lowest throughput values is approximately 20,814 records/second, highlighting meaningful differences in effective processing capacity.

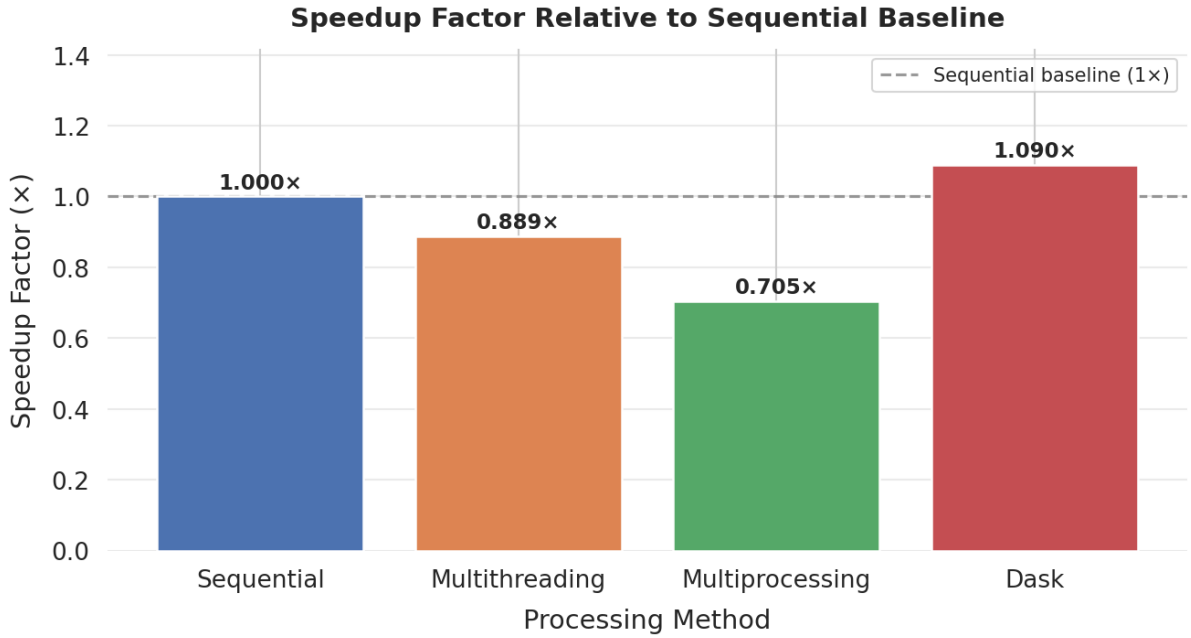

Chart 3 — Speedup Factor: The speedup chart relative to the Sequential baseline visually confirms that only Dask achieves a speedup greater than 1.0 (1.090×). Both Multithreading (0.889×) and Multiprocessing (0.705×) fall below the baseline, with Multiprocessing showing the most significant overhead penalty.

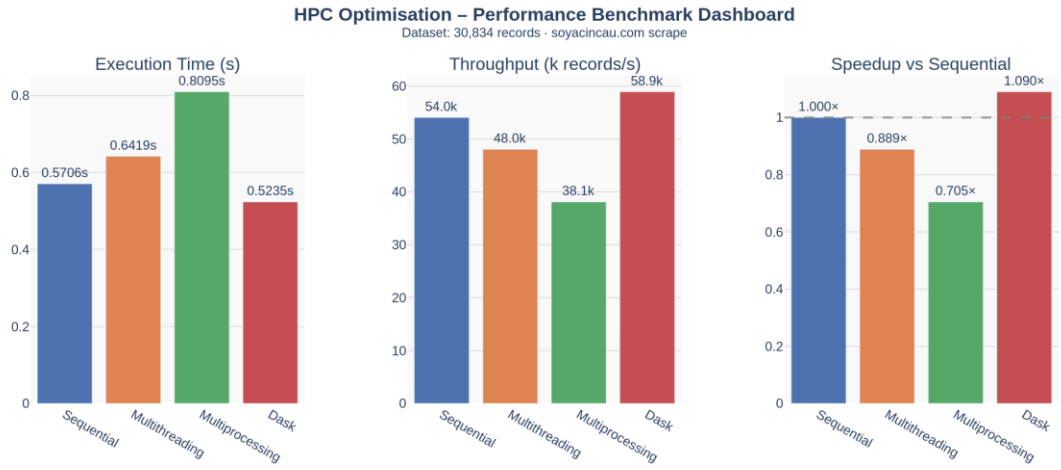

Chart 4 — Composite Dashboard: The Plotly interactive dashboard consolidates all three metrics in a side-by-side panel view with hover tooltips, enabling interactive exploration of the benchmark results.

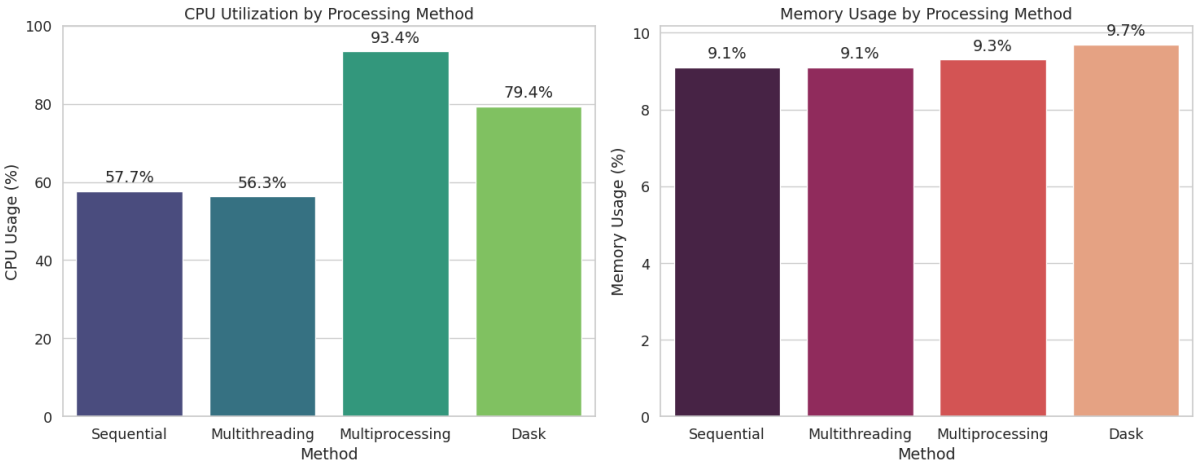

Chart 5 shows that dask consistently maintains the lowest execution time across all volumes, demonstrating superior scalability and efficiency for this pipeline. In contrast, Sequential and Multithreading methods exhibit a steep, linear increase in execution time as the data volume grows. Notably, while Multiprocessing incurs the highest initial overhead—making it the slowest method for smaller datasets—its execution time grows at a much flatter rate.

6.4 Statistical Interpretation

The benchmark results demonstrate several important principles of HPC system design:

- Dask's lazy evaluation and partition-aware task scheduler provide the best performance for in-memory pandas-style transformations even on a single node. Its 9.0% speedup over sequential processing is modest but consistent, and is expected to grow substantially at larger dataset scales where parallelism benefits overcome fixed overhead costs.
- Multithreading underperforms the sequential baseline due to Python's GIL, which prevents parallel CPU execution on CPU-bound string manipulation tasks. Thread management overhead adds latency without providing any CPU parallelism benefit.
- Multiprocessing incurs the heaviest overhead penalty due to inter-process serialization costs: each worker process requires its DataFrame partition to be serialized via pickle for inter-process communication, spawning separate Python interpreter instances adds startup latency, and result collection introduces further synchronization costs. For a 30,834-record in-memory dataset, these fixed costs dominate the execution time.
- The throughput standard deviation across all four methods is approximately 8,965 records/second, indicating meaningful variation in processing efficiency across techniques.

- All four methods complete processing within one second, confirming that the chosen dataset size is appropriate for a controlled benchmarking scenario. At 10× or 100× the current scale, the performance ranking would be expected to shift in favor of Dask and potentially Multiprocessing as parallelism benefits become dominant.

7.0 Challenges and Limitations

Several challenges and limitations were encountered during the execution of this project:

- Single-run benchmarks: Each processing method was timed over a single execution. Without repeated trials (a minimum of 30 runs) and statistical confidence intervals, execution time measurements are susceptible to noise from OS scheduling, garbage collection pauses, and Colab's shared resource environment. Future benchmarks should report mean execution times and standard deviations across multiple runs.
- In-memory dataset only: At 30,834 records, the dataset fits comfortably within RAM. The performance advantages of distributed engines such as Dask and Spark become significantly more pronounced at dataset sizes of millions of records where data must be partitioned across disk or cluster nodes.
- Google Colab single-node constraint: Colab provides a single shared CPU environment. True multiprocessing benefits require multiple physical CPU cores. The results obtained here would differ significantly on a local multi-core machine or a cloud-hosted multi-node cluster.
- Absence of memory profiling: Although psutil was imported in the optimization notebook, peak resident set size (RSS) memory consumption was not captured per method. Memory ceiling peaks are therefore absent from this iteration's performance evaluation, representing a gap relative to the assignment's full requirements.
- Homogeneous workload: All methods processed the same lightweight pandas string-transformation pipeline. Mixed workloads incorporating network I/O (such as the web crawling stage) would produce different performance rankings, likely favouring multithreading for its I/O concurrency benefits.
- Dataset size below 100,000 records: The collected dataset of 30,834 records falls short of the 100,000-record target specified in the assignment brief. This was due to the limited historical article volume available on the SoyaCincau platform at the time of crawling. The crawler architecture and pipeline are designed to scale; increasing coverage would require crawling over a longer period or targeting additional API endpoints.

8.0 Conclusion and Future Work

This project successfully delivered a complete end-to-end high-performance data pipeline for large-scale web crawling, cleaning, and processing. A functional web crawler was developed to extract structured article metadata from the SoyaCincau Malaysian technology news website by interfacing directly with its internal JSON REST API. The crawler collected 30,834 records across five structured fields, stored progressively in JSON format to prevent data loss.

A rigorous data cleaning pipeline was implemented to standardize string fields, validate URLs, parse and reformat dates to ISO 8601 standard, and remove any malformed records. The cleaned dataset of 30,834 records was exported to CSV and served as the input to performance benchmarking experiments.

Three HPC optimization techniques were implemented and benchmarked against a sequential baseline: Multithreading, Multiprocessing, and Dask distributed computing. The benchmark results demonstrated that Dask achieved the best overall performance with a throughput of 58,905 records per second and a 9.0% speedup over the sequential baseline, attributable to its lazy evaluation model and lightweight task graph scheduler. Both Multithreading and Multiprocessing underperformed the sequential baseline for this CPU-bound, in-memory workload due to the Python GIL and inter-process serialization overhead respectively.

Future work directions include:

1. Repeating benchmarks over 30+ trials with statistical significance testing (Welch's t-test) to validate performance differences rigorously.
2. Scaling experiments at 100,000, 500,000, and 1,000,000 records to identify the crossover point where distributed processing genuinely outperforms sequential execution.
3. Integrating memory profiling using tracemalloc or memory_profiler to capture peak heap usage alongside wall-clock time.
4. Replacing synchronous HTTP requests in the crawler with aiohttp and asyncio to introduce asynchronous I/O and measure end-to-end pipeline latency improvements.
5. Deploying the full pipeline on a multi-node Dask cluster (e.g. AWS EMR or Google Dataproc) to demonstrate true horizontal scalability across distributed infrastructure.
6. Extending the dataset to meet and exceed 100,000 records by crawling over a longer period or incorporating additional SoyaCincau content categories.

9.0 Reference

Presentation Video: https://youtu.be/jH9ZDoExayg